Kubernetes & Minikube on AWS EC2

Complete Guide: Provisioning EC2, Driver Setup, Cluster Init, Service Exposure & Browser Access

AWS Cloud | EC2 t3.medium / Ubuntu 24.04 | Security Groups & NodePort

Target Architecture

- **AWS EC2:** Ubuntu 24.04 / 22.04 LTS (`t3.medium` / 2 vCPU, 4GB RAM)
- **Storage:** 25-30 GB gp3 EBS Volume
- **Runtime:** Docker Engine (Minikube container driver)
- **Ingress:** NodePort `30080` open in EC2 Security Group

Security Group Inbound Rules Required

- **SSH:** Port 22 → Source: `My IP`
- **Custom TCP:** Port 30080 → Source: `0.0.0.0/0` (App NodePort)
- **HTTP:** Port 80 → Source: `0.0.0.0/0` (Optional / Proxy)

Phase 1: AWS EC2 Instance Provisioning

Step 1.1: Launch the EC2 Instance

In the AWS Management Console, navigate to **EC2** → **Launch Instances** with these specifications:

- **Name:** `k8s-minikube-lab`
- **AMI:** Ubuntu Server 24.04 LTS (HVM), SSD Volume Type (x86_64)
- **Instance Type:** `t3.medium` (Minikube requires at least 2 vCPUs and 2-4 GB memory; `t2.micro` will fail to initialize).
- **Key Pair:** Select or generate a new key pair (e.g., `k8s-key.pem`).
- **Storage:** Change root volume from 8 GB to **25 GB gp3** (cluster images require disk overhead).

Step 1.2: Configure Security Group Inbound Rules

Attach a security group with the following inbound rules:

Type	Protocol	Port Range	Source	Purpose
SSH	TCP	22	Your IP (or 0.0.0.0/0)	Instance Terminal Access
Custom TCP	TCP	30080	0.0.0.0/0	Access Nginx NodePort from Browser

Step 1.3: Connect via SSH

Set correct permissions on your private key and connect to the public IP of your EC2 instance:

```
chmod 400 k8s-key.pem
ssh -i "k8s-key.pem" ubuntu@<EC2-PUBLIC-IP>
```

Phase 2: Docker, Kubectl & Minikube Installation

Step 2.1: Update System & Install Docker Engine

```
# Update package index
sudo apt update && sudo apt upgrade -y
sudo apt install -y curl wget apt-transport-https ca-certificates gnupg lsb-release

# Add Docker GPG key and repo
sudo install -m 0755 -d /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor -o /etc/apt/keyrings/
docker.gpg
sudo chmod a+r /etc/apt/keyrings/docker.gpg

echo "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg] https://
download.docker.com/linux/ubuntu $(lsb_release -cs) stable" | sudo tee /etc/apt/sources.list.d/
docker.list > /dev/null

sudo apt update && sudo apt install -y docker-ce docker-ce-cli containerd.io

# Allow ubuntu user to execute docker without sudo
sudo usermod -aG docker $USER
newgrp docker
docker ps
```

Step 2.2: Install Kubectl CLI

```
curl -LO "https://dl.k8s.io/release/$(curl -L -s https://dl.k8s.io/release/stable.txt)/bin/linux/amd64/
kubectl"
sudo install -o root -g root -m 0755 kubectl /usr/local/bin/kubectl
kubectl version --client
```

Step 2.3: Install Minikube Binary

```
curl -LO https://storage.googleapis.com/minikube/releases/latest/minikube-linux-amd64
sudo install minikube-linux-amd64 /usr/local/bin/minikube
rm minikube-linux-amd64
minikube version
```

Phase 3: Cluster Startup & AWS Networking Specifics

Step 3.1: Start the Minikube Cluster

```
minikube start --driver=docker --cpus=2 --memory=3072
```

> **AWS / Cloud Note:** Minikube runs inside a Docker container on EC2. Its internal IP (e.g., `192.168.49.2`) is isolated from your local laptop and the public internet. Accessing the app requires port forwarding or NodePort binding to the EC2 host.

Step 3.2: Verify Cluster Health

```
minikube status
kubectl get nodes
kubectl cluster-info
```

Phase 4: Deploying Nginx Application & Exposing on EC2

Step 4.1: Create & Apply Deployment Manifest

```
cat <<EOF > nginx-deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web-app
  labels:
    app: web-app
spec:
  replicas: 2
  selector:
    matchLabels:
      app: web-app
  template:
    metadata:
      labels:
        app: web-app
    spec:
      containers:
      - name: nginx
        image: nginx:1.25-alpine
        ports:
        - containerPort: 80
EOF

kubectl apply -f nginx-deployment.yaml
kubectl get pods -l app=web-app
```

Step 4.2: Create NodePort Service with Fixed Port (30080)

```
cat <<EOF > nginx-service.yaml
apiVersion: v1
kind: Service
metadata:
  name: web-app-svc
spec:
  type: NodePort
  selector:
    app: web-app
  ports:
    - protocol: TCP
      port: 80
      targetPort: 80
      nodePort: 30080
EOF

kubectl apply -f nginx-service.yaml
kubectl get svc web-app-svc
```

Step 4.3: Expose Minikube to EC2 Public IP

Because the Minikube Docker container network is isolated from the EC2 host interface, run `kubectl port-forward` bound to `0.0.0.0`:

```
# Bind port 30080 on all EC2 interfaces (0.0.0.0) in background mode:
kubectl port-forward --address 0.0.0.0 svc/web-app-svc 30080:80 > /dev/null 2>&1 &
```

Step 4.4: Verify from Terminal and Web Browser

```
# Verification 1: Test locally on EC2 instance
curl http://localhost:30080

# Verification 2: Open in your laptop's browser:
# http://<YOUR-EC2-PUBLIC-IP>:30080
```

Phase 5: Clean Up AWS Resources

Prevent unnecessary AWS charges when finished:

```
# 1. Inside EC2: Delete Kubernetes objects and stop Minikube
kubectl delete -f nginx-service.yaml
kubectl delete -f nginx-deployment.yaml
minikube stop && minikube delete

# 2. In AWS Console: Terminate the EC2 instance
# EC2 Dashboard → Instances → Select Instance → Instance State → Terminate
```